

Linguaggi di Programmazione e Modelli Computazionali

Esame del 9 Gennaio 2014

Esercizio 1:

Progettare un automa a stati finiti deterministico che riconosce il linguaggio delle stringhe binarie che contengono un numero pari di 0 oppure che interpretate come numero in base 2 rappresentano un numero multiplo di 5.

Esercizio 2:

Definire una grammatica libera dal contesto che genera il linguaggio delle stringhe binarie con un ugual numero di 0 ed 1.

Esercizio 3:

Si consideri il linguaggio delle stringhe di parentesi tonde bilanciate del tipo ww (cioè costituite dalla concatenazione di due stringhe identiche). Classificare il linguaggio dicendo se è un linguaggio regolare, libero, ricorsivo, ricorsivamente enumerabile, o nemmeno ricorsivamente enumerabile. Giustificare la risposta.

Esercizio 4:

Si consideri il seguente linguaggio sull'alfabeto $\{0,1,2\}$:
 $L = \{x2y \mid x \text{ e } y \text{ sono numeri binari tali che } L(M_x) \text{ e } L(M_y) \text{ hanno una stringa in comune}\}$
dove M_x e M_y sono la x -esima e la y -esima macchina di Turing secondo una qualche enumerazione delle macchine di Turing. Dire se L è ricorsivo, ricorsivamente enumerabile, o nemmeno ricorsivamente enumerabile. Giustificare la risposta.

Esercizio 5:

Descrivere come si può ottenere un DFA da un ε -NFA.

Esercizio 6:

Si consideri la seguente grammatica:

If $\rightarrow$ *if c then* Block Else

Else $\rightarrow$ *else* ElseBlock | ε

Block $\rightarrow$ *stm* | ε

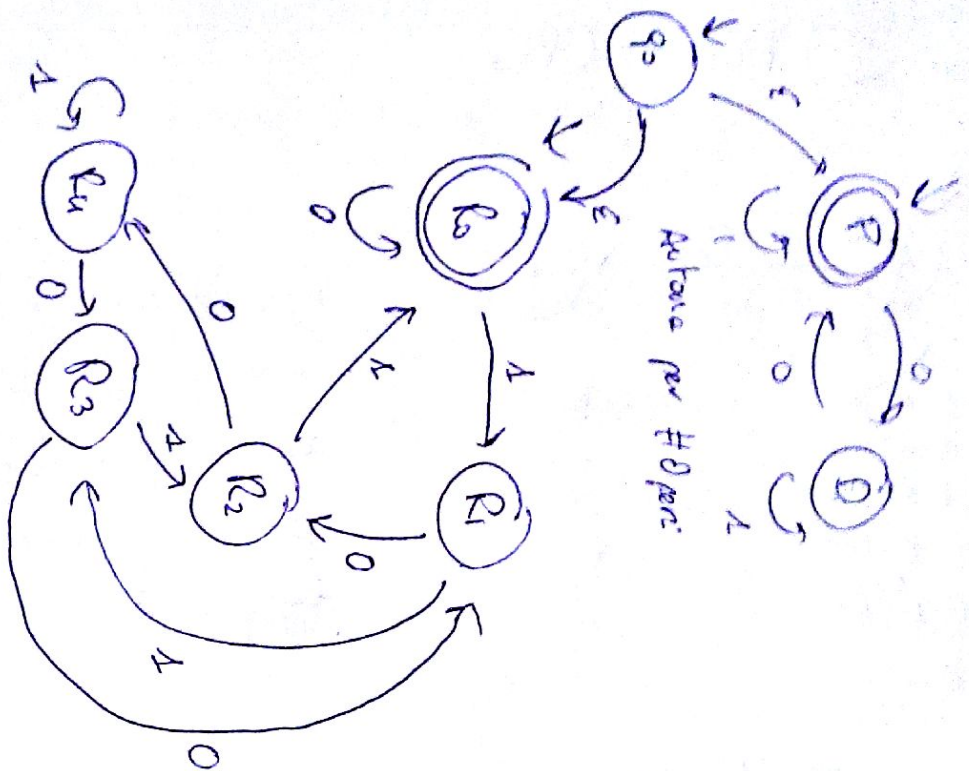
ElseBlock $\rightarrow$ If | Block

Dire se si tratta di grammatica di tipo LL(1) e nel caso presentare la corrispondente tabella di parsing.

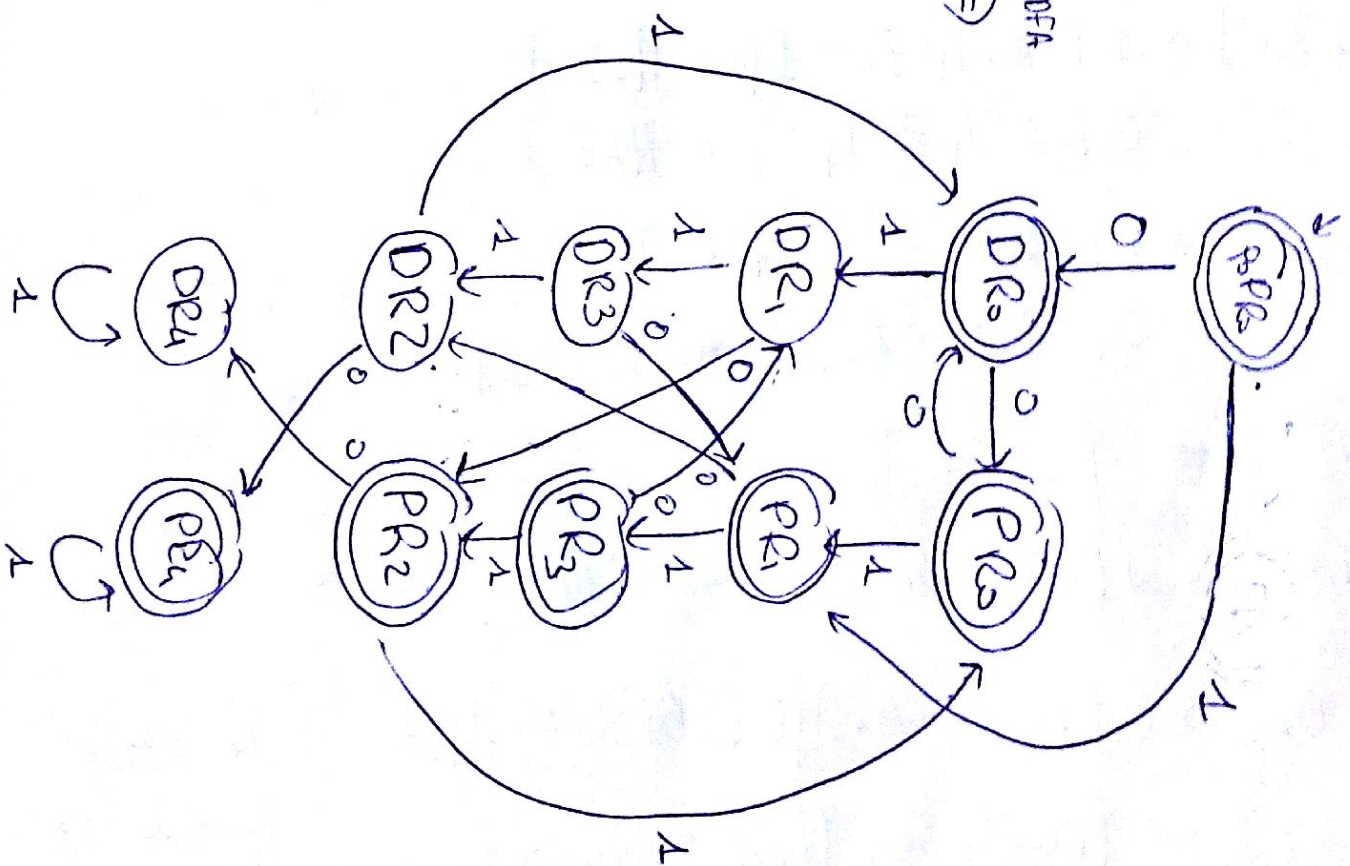
Esercizio 7:

Si consideri un sistema concorrente costituito da un processo e una variabile booleana inizialmente non inizializzata. Il processo può scrivere i valori TRUE o FALSE nella variabile condivisa oppure leggere la variabile stessa. Il sistema deve essere progettato in modo tale che, dopo un'operazione che scrive TRUE (rispettivamente FALSE) nella variabile, tutte le successive operazioni di lettura non restituiscano il valore FALSE (rispettivamente TRUE) a meno che nel frattempo la variabile non sia stata riscritta. Descrivere tale sistema usando un'algebra di processi e formalizzare tale proprietà in LTL.

Ex. 1) $\epsilon\text{-NFA} \rightarrow \text{DFA}$



$\epsilon\text{-NFA} \rightarrow \text{DFA}$

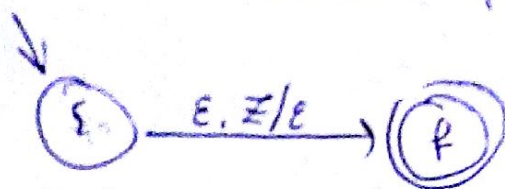


ESAME

08/01/2014

Es. 2) $PM \rightarrow CFG$

Automazione per stato finale o pile vuote



1 Z/AZ

1 A/AA

1 B/E

0 Z/BZ

0 B/BB

0 A/E

configurazioni

ii

if

fi

ff

ogni 1 aggiunge un A o toglie un B. ogni 0 aggiunge un B o toglie un A.

Accetto se alla fine dell'input

l'input è

$S \rightarrow [iZi][iZf]$ } dalla definizione del procedimento

$[iZi] \rightarrow 1 [iAi][iZi] | 1 [iAf][fZi] | 0 [iBi][iZi] | 0 [iBf][fZi]$ } da $1Z/AZ$

$[iZf] \rightarrow 1 [iAi][iZf] | 1 [iAf][fZf] | 0 [iBi][iZf] | 0 [iBf][fZf] | \epsilon$ } da $1Z/AZ$
da $\epsilon Z/E$

$[iAi] \rightarrow 0 | 1 [iAi][iAi] | 1 [iAf][fAi]$ } da $1A/AA$
 $[iAf] \rightarrow 1 [iAi][iAf] | 1 [iAf][fAf]$ } da $0A/E$

$[iBi] \rightarrow 0 [iBi][iBi] | 0 [iBf][fBi] | 1$ } da $0B/BB$
 $[iBf] \rightarrow 0 [iBi][iBf] | 0 [iBf][fBi]$ } da $1B/E$

Non ci sono regole che abbiano nel LHS una tripletta che inizia con f
 $\Rightarrow$ sono tutti simboli non garantiti e posso togliere tutte le regole di questo tipo

$Z = [iZi]$ $A = [iAi]$ $B = [iBi]$

$Z' = [iZf]$ $A' = [iAf]$ $B' = [iBf]$

La grammatica è quindi:

$$S \rightarrow \varepsilon \mid z'$$

$$\cancel{z \rightarrow 1Az \mid 0Bz} \text{ (non generante)}$$

$$z' \rightarrow 1Az' \mid 0Bz' \mid \varepsilon$$

$$A \rightarrow 0 \mid 1AA$$

$$\cancel{A' \rightarrow 1AA'} \text{ (non generante)}$$

$$B \rightarrow 1 \mid 0BB$$

$$\cancel{B' \rightarrow 0BB'} \text{ (non generante)}$$

rinomino $z' = S$

$$S \rightarrow 1AS \mid 0BS \mid \varepsilon$$

$$A \rightarrow 0 \mid 1AA$$

$$B \rightarrow 1 \mid 0BB$$

Seq. binarie
con
 $\#1 = (\#0 + 1)$

Sequenza
binaria
con $\#1 = (\#0 - 1)$

Es 3) Classificazione linguaggio

$$L = L_1 \cap L_2$$

$L_1 = \{ \text{linguaggio delle parentesi tonde bilanciate} \}$ CFL

$L_2 = \{ \text{linguaggio delle stringhe del tipo } ww \text{ con } w \in \{ (,) \}^* \}$ Ricorso

Utilizzo: è ricorsivo poiché intersezione di un CFL con un ricorsivo
(i CFL in quanto tali sono ricorsivi ed questi ultimi sono
chiusi rispetto all'intersezione)

Potenza 1) Dimostrare che L non è CFL:

Se L fosse CFL per P'dei liben $\exists n: \forall z \in L: |z| \geq n, z = z_1 z_2 z_3 z_4$

$$\textcircled{I} |z_1 z_3 z_4| \leq n$$

$$\textcircled{II} |z_1 z_4| \geq 1$$

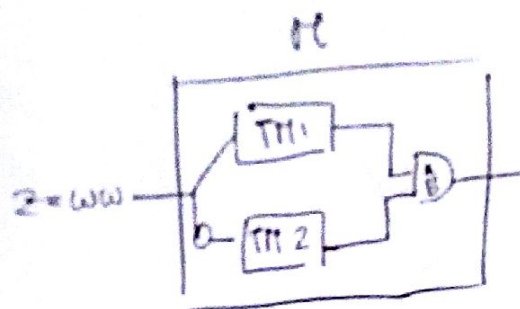
$$\textcircled{III} z_i = z_1 (z_2)^i z_3 (z_4)^i z_5 \in L \forall i \in \mathbb{N}$$

$$\text{Considero } z = \underbrace{\left(\underbrace{}_{g_1} \right)^n \underbrace{\left(\underbrace{}_{g_2} \right)^n}_{g_3} \underbrace{\left(\underbrace{}_{g_4} \right)^n}_{g_5}$$

Se scelgo $z_2 \in g_1$ sono costretto a scegliere z_4 o in g_1 , o in g_2 o a cavallo
tra i due. In ogni caso $\forall i \neq 1 z_i \notin L$ poiché z_i non sarebbe composto da due
stringhe uguali concatenate. Ragionamenti analoghi per ogni altra scelta di z_2

$\Rightarrow L$ non è CFL

Piano 2) Dimostrazione di $FTM M \mid L(M) = L$



- Considero M composta da due TM M_1 e M_2 . M accetta se qualcuno dei due.
- M_1 verifica che $Z \in L_1$. Ha due nastri. Su uno contiene Z e ~~inizialmente~~ l'altro viene usato come uno stack, quindi inizialmente contiene solo un $\#$ (il simbolo). Scorrendo l'input M_1 fa push di A ogni volta che legge (dal primo nastro) A o Z se seconda. Se incontra $)$ e A fa la pop di A e prende, mentre se incontra $)$ e Z termina rifiutando. Se alla fine dell'input allo stack rimane solo Z termina accettando.
- M_2 prova non-deterministicamente tutti i truncamenti di Z , copiando se solo secondo truncamento all'inizio di un secondo nastro e cancellandolo dal primo. ~~Il~~ Finito la copia torna con le testine all'inizio di ambo i nastri e confronta ogni coppia di simboli. Alla prima differenza termina rifiutando. Se uno dei due nastri finisce prima dell'altro termina rifiutando. ~~Se alla fine~~ Se incontra $\#$ su ambo i nastri termina accettando.

Se M_1 o M_2 terminano su ogni input quindi M lo accetta

ES. 4) Classificazione Pumpingio

Due interpretazioni per L:

- (I) $L(M_x)$ e $L(M_y)$ hanno almeno una stringa in comune
- (II) $L(M_x)$ e $L(M_y)$ hanno solo una stringa in comune

~~Caso I~~ Caso (I)

~~Restante~~ Esiste una TM M che prende in ingresso x^2y con $x, y \in \{0,1\}^*$ e considera tali stringhe gli input di due TTM nelle tabelle di accettazione

	TM_1	TM_2	...	TM_j	...
w_1					
w_2					
$\vdots$					
w_i	-	-	-	-	-

Per dire se $|L(M_x) \cap L(M_y)| \geq 1$ M deve scansionare le colonne x e y alla ricerca di un i per cui

$$\langle i, x \rangle = \langle i, y \rangle = 1$$

Se esiste tale i , M lo trova in un tempo finito ed accetta -

Se non esiste M entra in un loop infinito poiché deve scansionare due colonne infinite -

$$x = 1 \Leftrightarrow w_i \in L(M_j)$$

$$x = 0 \Leftrightarrow w_i \notin L(M_j)$$

TABELLA DI ACCETTAZIONE

Ne consegue che L è RE ma non R

Alternativamente si può ragionare così:

~~M ha due nastri. Non deterministicamente sceglie una stringa w_i e simula sul primo nastro M_x con input w_i e sul secondo M_y con lo stesso input. Se è rifiuto lo accetta. Se è accettazione, M accetta se M_x e M_y accettano entrambe, rifiuta se anche se solo rifiuta. Abbiamo ridotto L_x e $L_y \Rightarrow L$ non è ricorsivo.~~

~~M accetta se una delle istanze accetta per una qualche stringa $\Rightarrow L$ è RE.~~

Tuttavia M deve verificare che ci sia una ed una sola i per cui $\langle x, i \rangle = \langle y, i \rangle = 1$

Caso (II)

Consideriamo lo stesso M di cui sopra. Ci sono 3 sotto casi

IIa) $|L_x \cap L_y| = 0 \Rightarrow M$ andrà alla ricerca di un i che non esiste entrando in un loop infinito

IIb) $|L_x \cap L_y| = 1 \Rightarrow$ ~~Una volta trovato i , M dovrà comunque verificare che non ci siano altre i per cui $\langle x, i \rangle = \langle y, i \rangle = 1$ e, non essendocene, entrerà in un loop infinito~~

IIc) $|L_x \cap L_y| > 1 \Rightarrow M$ terminerà rifiutando appena trova la seconda stringa.

Dato che non riesco a determinare né l'appartenenza di x a L né la non appartenenza ne deduco che L è sia non RE.

ES 5) Teorema $ENFA \rightarrow DFA$

Sia E un ENFA $| E = (Q_E, \Sigma, \delta_E, q_0, F_E)$

Vogliamo ottenere un DFA $D | D = (Q_D, \Sigma, \delta_D, \epsilon_{close}(q_0), F_D)$
dove $Q_D \subseteq 2^{Q_E}$

e $\epsilon_{close}(p) \triangleq \{ q \in Q_E | q \text{ è raggiungibile da } p \text{ con una sequenza di } \epsilon \text{ transizioni} \}$

$$F_D = \{ S \in Q_D | S \cap F_E \neq \emptyset \}$$

$$\delta_D(S, a) = \bigcup_{t \in S} \left(\bigcup_{p \in \delta_E(t, a)} \epsilon_{close}(p) \right)$$

ES. 6) LL(1)

Rinomino:

If = I if = i then = t Block = B Else = E

else = e ElseBlock = X

La grammatica diventa

$I \rightarrow i c t B E$

$E \rightarrow e X | \epsilon$

$B \rightarrow s | \epsilon$

$X \rightarrow I | B$

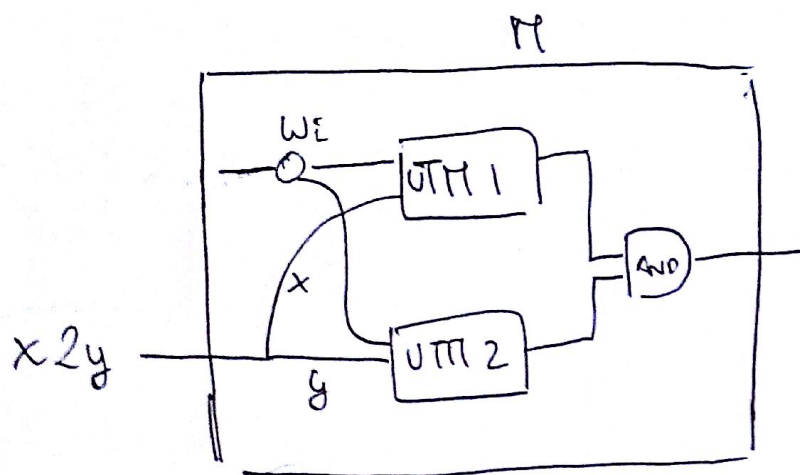
	first	follow
I	i	\$
E	e E	follow(I): \$
B	s E	first(E) = e follow(E): \$ \$e
X	i B E	follow(E): \$: \$

$\uparrow$ $\uparrow$
 $de X$ $de B$

	i	c	t	e	s	\$
I	ictBE					
E				eX		E
B				E	S	E
X	I				B	B

$\Rightarrow$ No conflict
 $\Downarrow$
 $G \in LL1$

Es. 4) Soluzione alternativa caso 1: $|L_x \cap L_y| \geq 1$



Considero M composta da due UTM: U_1 e U_2 .

M : Selezione non-deterministicamente una stringa $w\epsilon$ e poi
fa simulare M_x a U_1 con input $w\epsilon$ e M_y a U_2 con input $w\epsilon$
 M accetta & accetta entrambe.

Ho ridotto L_0 a $L = L_x \cap L_y \Rightarrow L$ non è ricorsivo.

Ho costruito un TM che accetta se $x2y \in L \Rightarrow L \in RE$.